

DevOps Interview Q&A Guide

Complete structured answers for 0–2 years DevOps / Cloud / SRE roles

[AWS](#) · [Kubernetes](#) · [Docker](#) · [Terraform](#) · [CI/CD](#) · [Linux](#) · [Monitoring](#) · [Security](#)

1. DevOps Fundamentals

Q: What is DevOps?

- DevOps is a combination of Development (Dev) and Operations (Ops).
 - It is a culture, set of practices, and tools that help teams build, test, and release software faster and more reliably.
 - DevOps breaks the wall between developers (who write code) and operations teams (who deploy and maintain systems).
 - The goal is continuous delivery of value to end users.
-

Q: Why was DevOps introduced?

- Traditional teams had a 'wall' between developers and operations — they worked in silos.
 - Developers wrote code and 'threw it over the wall' to Ops, causing delays and blame-shifting.
 - Releases were slow, manual, and error-prone.
 - DevOps was introduced to bridge this gap and make delivery faster, automated, and collaborative.
-

Q: What problems does DevOps solve?

- Slow software releases — DevOps automates build, test, and deploy steps.
 - Manual errors — automation removes human mistakes from repetitive tasks.
 - Communication gaps — Dev and Ops now share responsibility.
 - Long feedback loops — CI/CD gives instant feedback on code quality.
 - Inconsistent environments — Docker/IaC ensures 'works on my machine' is never an issue.
-

Q: What are the core principles of DevOps?

- Collaboration — Dev and Ops teams work together.
 - Automation — automate repetitive tasks (builds, tests, deployments).
 - Continuous Integration & Delivery — frequently integrate and ship code.
 - Monitoring & Feedback — continuously monitor and improve systems.
 - Infrastructure as Code — manage infrastructure the same way as application code.
-

Q: Explain the DevOps lifecycle.

- Plan — define features and tasks (Jira, Trello).
 - Code — developers write and commit code (Git, GitHub).
 - Build — code is compiled and packaged (Maven, Docker).
 - Test — automated tests run (JUnit, Selenium, SonarQube).
 - Release — approved build is prepared for deployment.
 - Deploy — release is pushed to staging or production (Jenkins, GitHub Actions, Helm).
 - Operate — application runs in production (Kubernetes, AWS).
 - Monitor — track performance and errors (Prometheus, Grafana, Datadog).
-

Q: What is Infrastructure as Code (IaC)?

- IaC means managing and provisioning infrastructure using code files instead of manual clicks.
 - You write configuration files (e.g., Terraform .tf files) that define servers, networks, databases.
 - These files are stored in Git and can be versioned, reviewed, and reused.
 - Benefits: consistent environments, fewer human errors, faster provisioning.
 - Tools: Terraform, AWS CloudFormation, Ansible, Pulumi.
-

Q: What is Continuous Integration (CI)?

- CI is the practice of frequently merging developer code into a shared repository.
 - Every merge triggers an automated pipeline that builds and tests the code.
 - Catches bugs early, before they reach production.
 - Example: Developer pushes code → Jenkins/GitHub Actions runs tests → result shown immediately.
-

Q: What is Continuous Delivery vs Continuous Deployment?

- Continuous Delivery — code is always in a deployable state; a human manually approves the final push to production.
 - Continuous Deployment — every change that passes all tests is automatically deployed to production without manual approval.
 - Continuous Delivery = automated up to production gate. Continuous Deployment = fully automated including production.
-

2. Linux

Q: What is Linux?

- Linux is a free, open-source operating system kernel created by Linus Torvalds in 1991.
 - It is widely used in servers, cloud infrastructure, containers, and DevOps environments.
 - Popular distributions: Ubuntu, CentOS, Amazon Linux, RHEL, Debian.
-

Q: Explain file permissions in Linux.

- Each file/directory has three permission groups: Owner, Group, Others.
 - Three permission types: r (read=4), w (write=2), x (execute=1).
 - Example: `chmod 755 file.sh` means Owner=rwx(7), Group=rx(5), Others=rx(5).
 - `chown user:group file` — changes the owner of a file.
 - `ls -l` shows permissions like: `-rwxr-xr-x`
-

Q: Explain grep, awk, sed, find.

- `grep` — searches for patterns in files. Example: `grep 'error' app.log`
 - `awk` — processes and formats text. Example: `awk '{print $1}' file.txt` prints first column.
 - `sed` — stream editor to find and replace text. Example: `sed 's/old/new/g' file.txt`
 - `find` — locates files/directories. Example: `find /var/log -name '*.log' -mtime -1`
-

Q: What is a daemon? What is systemd?

- Daemon — a background process that runs continuously (e.g., `nginx`, `sshd`, `crond`).
 - `systemd` — the init system that manages services (daemons) on modern Linux.
 - Commands: `systemctl start/stop/restart/status nginx`
 - `systemctl enable nginx` — starts `nginx` automatically on boot.
-

Q: Troubleshooting: Server CPU is at 100%. How do you investigate?

- Run `top` or `htop` to see which process is consuming CPU.
 - Use `ps aux --sort=-%cpu | head` to list top CPU consumers.
 - Use `pidstat` or `mpstat` for per-CPU stats.
 - Check if a runaway process or infinite loop is the cause.
 - Kill the offending process if needed: `kill -9`
 - Check application logs for errors driving high CPU.
- *Always check logs + metrics together before killing a process in production.*
-

Q: Troubleshooting: Disk space is full.

- Check usage: `df -h` (shows disk usage per filesystem).
 - Find large files: `du -sh /* | sort -rh | head -20`
 - Common culprits: old Docker images, large log files, unused packages.
 - Clean Docker: `docker system prune -a`
 - Rotate or truncate logs: `logrotate` or `truncate -s 0 /var/log/app.log`
-

3. Networking

Q: What is TCP/IP? Difference between TCP and UDP?

- TCP/IP is the fundamental communication protocol suite of the internet.
 - TCP (Transmission Control Protocol) — reliable, ordered, connection-based. Used for HTTP, SSH, databases.
 - UDP (User Datagram Protocol) — fast, connectionless, no delivery guarantee. Used for DNS, video streaming, gaming.
 - Key difference: TCP guarantees delivery; UDP prioritises speed over reliability.
-

Q: What is DNS?

- DNS (Domain Name System) — translates human-readable domain names (google.com) into IP addresses (142.250.x.x).
 - Think of it as the 'phonebook of the internet'.
 - Types: A record (domain → IPv4), CNAME (alias), MX (mail), TXT (verification).
 - Commands: nslookup google.com or dig google.com
-

Q: What is a Load Balancer? What is a Reverse Proxy?

- Load Balancer — distributes incoming traffic across multiple servers to prevent overload.
 - Types: Application (Layer 7, HTTP/HTTPS) and Network (Layer 4, TCP/UDP).
 - Reverse Proxy — sits in front of backend servers and forwards client requests to them.
 - Nginx is commonly used as both a reverse proxy and load balancer.
 - AWS ALB (Application Load Balancer) is a managed load balancer service.
-

Q: What is a CIDR block? What is a subnet?

- CIDR (Classless Inter-Domain Routing) — a method of specifying IP address ranges.
 - Example: 10.0.0.0/16 means the first 16 bits are the network part → 65,536 IP addresses.
 - Subnet — a subdivision of a network. 10.0.1.0/24 gives 256 addresses.
 - In AWS VPC: /16 for VPC, /24 for subnets is a common pattern.
-

Q: Troubleshooting: Website is not accessible.

- Check if the server is reachable: ping
 - Check DNS resolution: nslookup
 - Check if the port is open: telnet 80 or curl -I http://
 - Check if the application is running: systemctl status nginx
 - Check firewall / security group rules — is port 80/443 open?
 - Check load balancer health checks in AWS console.
-

4. Git

Q: What is Git? What is GitHub?

- Git — a distributed version control system that tracks code changes locally.
 - GitHub — a cloud-based hosting platform for Git repositories with collaboration features.
 - Other alternatives: GitLab, Bitbucket.
-

Q: What is branching? What is merging? What is rebasing?

- Branching — creating an isolated copy of the codebase to work on a feature without affecting main.
 - Merging — combining changes from one branch into another (git merge feature-branch).
 - Rebasing — re-applies commits on top of another branch, creating a cleaner linear history (git rebase main).
 - Rebase vs Merge: rebase = clean history; merge = preserves full history.
-

Q: What is GitFlow? What is Trunk-Based Development?

- GitFlow — uses multiple long-lived branches: main, develop, feature/*, release/*, hotfix/*. Suited for scheduled releases.
 - Trunk-Based Development — all developers commit to a single main branch frequently (daily). Feature flags used for incomplete work. Suited for CI/CD and DevOps teams.
-

Q: How do you resolve Git conflicts?

- A conflict occurs when two branches modify the same line differently.
 - Git marks the conflict in the file with <<<<<, =====, >>>>>.
 - You manually edit the file to keep the correct version.
 - Then: git add → git commit to complete the merge.
 - Use git status to see which files are conflicted.
-

5. Docker

Q: What is Docker? Difference between Docker and a VM?

- Docker is a tool to package applications and their dependencies into lightweight portable containers.
 - VM — includes a full OS on top of a hypervisor. Heavy (~GBs), slow to start.
 - Container — shares the host OS kernel. Lightweight (~MBs), starts in seconds.
 - Key advantage: 'build once, run anywhere' — same container runs on any machine.
-

Q: What is a Dockerfile? Key instructions?

- Dockerfile — a text file with step-by-step instructions to build a Docker image.
 - FROM — base image (e.g., FROM python:3.11-slim).
 - RUN — executes commands during build (e.g., RUN pip install -r requirements.txt).
 - COPY — copies files into the image.
 - ENV — sets environment variables.
 - EXPOSE — documents which port the container listens on.
 - CMD — default command to run when container starts (can be overridden).
 - ENTRYPOINT — fixed command that always runs (harder to override).
-

Q: Difference between CMD and ENTRYPOINT?

- ENTRYPOINT — defines the main executable. Always runs. Example: ENTRYPOINT ['python', 'app.py']
 - CMD — provides default arguments; can be overridden at runtime.
 - Used together: ENTRYPOINT is the command, CMD is the default argument.
 - Example: docker run myimage --debug overrides CMD but not ENTRYPOINT.
-

Q: How do you optimise Docker images?

- Use a slim base image: python:3.11-slim instead of python:3.11.
 - Use multi-stage builds — build in one stage, copy only the artifact to a smaller final image.
 - Combine RUN commands using && to reduce layers.
 - Add a .dockerignore file to exclude unnecessary files (like .git, __pycache__).
 - Use specific image tags, not 'latest', for reproducibility.
-

Q: Troubleshooting: Container exits immediately.

- Run: docker logs to see the error.
 - Run interactively: docker run -it myimage /bin/sh to debug.
 - Common causes: application crash, missing environment variable, wrong CMD/ENTRYPOINT.
 - Check if the process inside is a foreground process — containers stop when the main process exits.
-

6. Kubernetes

Q: What is Kubernetes?

- Kubernetes (K8s) is an open-source container orchestration platform.
 - It automates deployment, scaling, load balancing, and self-healing of containerised applications.
 - Manages clusters of nodes; ensures containers run in the desired state.
-

Q: What is a Pod? A Deployment? A Service?

- Pod — the smallest deployable unit in Kubernetes. Contains one or more containers sharing the same network and storage.
 - Deployment — manages a set of identical Pods. Handles rolling updates and rollbacks. You define desired replicas.
 - Service — provides a stable network endpoint (IP + DNS) to access Pods. Types: ClusterIP, NodePort, LoadBalancer.
-

Q: What is an Ingress? A ConfigMap? A Secret?

- Ingress — manages external HTTP/HTTPS traffic to Services. Acts as a reverse proxy / API gateway inside K8s.
 - ConfigMap — stores non-sensitive configuration data (env vars, config files) as key-value pairs.
 - Secret — stores sensitive data (passwords, tokens, keys) in base64-encoded form. Should be integrated with Vault in production.
-

Q: What is HPA (Horizontal Pod Autoscaler)?

- HPA automatically scales the number of Pod replicas based on CPU/memory usage or custom metrics.
 - Example: if CPU > 70%, HPA adds more Pods; if CPU drops, it scales down.
 - Configured via: `kubectl autoscale deployment myapp --cpu-percent=70 --min=2 --max=10`
-

Q: What is RBAC in Kubernetes?

- RBAC (Role-Based Access Control) controls who can do what in a Kubernetes cluster.
 - Role — defines permissions within a namespace.
 - ClusterRole — defines permissions cluster-wide.
 - RoleBinding — assigns a Role to a user/service account.
 - Principle of least privilege: give only the minimum required permissions.
-

Q: Troubleshooting: Pod stuck in Pending.

- Run: `kubectl describe pod` — check Events section.
 - Common causes: insufficient CPU/memory on nodes, no available nodes matching node selector, PVC not bound.
 - Check node resources: `kubectl describe nodes | grep -A5 Allocated`
-

Q: Troubleshooting: Pod in CrashLoopBackOff.

- The container starts, crashes, and Kubernetes keeps restarting it.
 - Check logs: `kubectl logs --previous`
 - Common causes: application error, missing env variable, wrong config, OOM kill.
 - Describe the pod: `kubectl describe pod` for exit codes.
-

Q: Troubleshooting: ImagePullBackOff.

- Kubernetes cannot pull the container image from the registry.
 - Check the image name and tag for typos.
 - Verify the image exists in the registry.
 - Check if an `imagePullSecret` is needed for private registries.
 - Verify node network can reach the registry.
-

7. AWS

Q: What is IAM? Difference between User, Role, and Policy?

- IAM (Identity and Access Management) — controls who can access AWS resources and what they can do.
 - User — represents a person or application with long-term credentials (access keys).
 - Role — a temporary identity assumed by AWS services, EC2 instances, or applications. No long-term credentials.
 - Policy — a JSON document that defines permissions. Attached to users, groups, or roles.
 - Best practice: never use root; use roles for services; grant least privilege.
-

Q: What is IRSA?

- IRSA (IAM Roles for Service Accounts) — allows Kubernetes Pods running in EKS to assume IAM Roles.
 - Instead of using node-level IAM permissions, each Pod gets a fine-grained role via a service account.
 - Enables least-privilege access for individual workloads inside EKS.
-

Q: What is a VPC? Public vs Private Subnet? NAT Gateway?

- VPC (Virtual Private Cloud) — your own isolated network in AWS.
 - Public Subnet — has a route to the Internet Gateway. Resources get public IPs. Used for load balancers, bastion hosts.
 - Private Subnet — no direct internet access. Used for application servers, databases.
 - NAT Gateway — allows resources in private subnets to initiate outbound internet connections (e.g., to download packages) without being publicly accessible.
-

Q: Security Group vs Network ACL (NACL)?

- Security Group — acts as a virtual firewall at the instance/ENI level. Stateful (return traffic is automatic). Allow rules only.
 - NACL — acts at the subnet level. Stateless (must define inbound AND outbound rules). Supports both allow and deny rules.
 - SGs are the first line of defence; NACLs add an extra layer.
-

Q: What is S3? EBS? EFS?

- S3 (Simple Storage Service) — object storage for files, backups, static websites. Virtually unlimited capacity.
 - EBS (Elastic Block Store) — block storage attached to a single EC2 instance. Like a hard drive. Used for OS, databases.
 - EFS (Elastic File System) — managed NFS file system mountable by multiple EC2 instances simultaneously.
-

Q: What is RDS? Multi-AZ vs Read Replica?

- RDS (Relational Database Service) — managed database service supporting MySQL, PostgreSQL, Oracle, SQL Server.
 - Multi-AZ — synchronous standby replica in another AZ for high availability and automatic failover.
 - Read Replica — asynchronous copy used to offload read traffic. Does NOT automatically failover.
 - Multi-AZ = HA/failover. Read Replica = performance/read scaling.
-

Q: What is EKS?

- EKS (Elastic Kubernetes Service) — AWS-managed Kubernetes control plane.
 - AWS manages the master nodes (API server, etcd). You manage worker nodes (EC2 or Fargate).
 - Integrates natively with IAM, ALB, CloudWatch, ECR.
-

8. Terraform & IaC

Q: What is Terraform? Key concepts?

- Terraform is an open-source IaC tool by HashiCorp to provision and manage cloud infrastructure.
 - Provider — plugin that interacts with a cloud API (e.g., AWS, Azure, GCP).
 - Resource — an infrastructure object to create (e.g., `aws_instance`, `aws_s3_bucket`).
 - State — a file (`terraform.tfstate`) that records the current real-world infrastructure.
 - Plan — shows what changes Terraform will make before applying.
 - Apply — executes the changes.
-

Q: Why use Remote State? What is S3 Backend + DynamoDB locking?

- Remote State — stores `terraform.tfstate` in a shared location (S3) so the whole team uses the same state.
 - S3 Backend — stores the state file in an S3 bucket (versioned for history).
 - DynamoDB Locking — prevents two people from running `terraform apply` at the same time (state locking).
 - Without locking, concurrent applies can corrupt the state file.
-

Q: What are Terraform Modules?

- Modules are reusable groups of Terraform resources, like functions in programming.
 - You define a module once (e.g., VPC module) and call it in multiple environments.
 - Promotes DRY (Don't Repeat Yourself) principle.
 - Source can be local path, Git repo, or Terraform Registry.
-

Q: Scenario: Someone manually deletes an AWS resource. What happens on the next Terraform plan?

- Terraform compares the state file with the real infrastructure.
- It detects the resource is missing (state says it exists, but reality says it doesn't).
- `terraform plan` shows the resource will be re-created.
- `terraform apply` re-creates the deleted resource.
- This is called 'Terraform drift detection'.

■ *This is why manual changes to Terraform-managed infrastructure are dangerous — always use Terraform to make changes.*

Q: What is `for_each`? What is `depends_on`?

- `for_each` — creates multiple instances of a resource from a map or set. More flexible than `count`.
 - Example: `for_each = var.subnets` creates one subnet resource per entry.
 - `depends_on` — explicitly tells Terraform to create one resource before another when the dependency isn't implicit.
-

9. CI/CD

Q: Explain CI/CD and your pipeline workflow.

- CI (Continuous Integration) — code pushed → automated build + test.
 - CD (Continuous Delivery/Deployment) — tested build → packaged → deployed to environment.
 - Typical pipeline stages: Source → Build → Test → Security Scan → Docker Build → Push to ECR → Deploy to EKS.
-

Q: What is a Jenkinsfile? Key concepts?

- Jenkinsfile — a Groovy-based script defining a Jenkins pipeline as code, stored in Git.
 - Stages — logical steps in the pipeline (Build, Test, Deploy).
 - Steps — actual commands within a stage (sh 'docker build', sh 'kubectl apply').
 - Agent — defines where the pipeline runs (any node, specific label, Docker container).
 - Credentials — secrets stored in Jenkins and injected via withCredentials block.
-

Q: What is a GitHub Actions Workflow?

- A YAML file in .github/workflows/ that defines automated workflows triggered by events.
 - Trigger: on: push, pull_request, schedule.
 - Job — a set of steps that run on a runner (ubuntu-latest, self-hosted).
 - Step — individual action or shell command.
 - Action — a reusable unit (e.g., actions/checkout, aws-actions/configure-aws-credentials).
-

Q: How do you manage secrets in CI/CD pipelines?

- Jenkins — store secrets in Jenkins Credentials Store. Access via withCredentials.
 - GitHub Actions — store in GitHub Repository Secrets. Access via \${ secrets.MY_SECRET }.
 - Never hardcode secrets in pipeline files or source code.
 - For Kubernetes — use Kubernetes Secrets or integrate with HashiCorp Vault / AWS Secrets Manager.
 - Rotate secrets regularly and audit access.
-

Q: What is SonarQube? What is Trivy?

- SonarQube — a static code analysis tool that checks code for bugs, code smells, security vulnerabilities, and code coverage.
 - Integrated in CI pipelines; fails the build if quality gates are not met.
 - Trivy — an open-source container image and filesystem vulnerability scanner.
 - Scans Docker images for known CVEs (Common Vulnerabilities and Exposures) before pushing to registry.
 - Both together implement DevSecOps — security embedded in the pipeline.
-

Q: How do you rollback a failed deployment?

- Kubernetes Deployment: `kubectl rollout undo deployment/myapp`
 - View history: `kubectl rollout history deployment/myapp`
 - Helm: `helm rollback`
 - Jenkins/GitHub Actions: re-trigger the pipeline with the last known good image tag.
 - Best practice: always tag images with Git commit SHA, never just 'latest'.
-

10. Monitoring & Observability

Q: Difference between Monitoring and Observability?

- Monitoring — tracking known metrics and alerting when they cross thresholds. Tells you WHAT is broken.
 - Observability — the ability to understand the internal state of a system from its external outputs (logs, metrics, traces). Tells you WHY it is broken.
 - Three pillars of Observability: Metrics, Logs, Traces.
-

Q: What is Prometheus? What is Grafana?

- Prometheus — an open-source metrics collection and alerting system. Scrapes metrics from targets via HTTP.
 - Stores time-series data. Queried using PromQL.
 - Grafana — a visualisation and dashboarding tool. Connects to Prometheus (and other sources) to show charts and graphs.
 - Alertmanager — part of Prometheus stack that routes alerts to Slack, PagerDuty, email.
-

Q: What is the ELK Stack?

- ELK = Elasticsearch + Logstash + Kibana.
 - Logstash / Filebeat — collects and ships logs from applications/servers.
 - Elasticsearch — stores and indexes logs for fast search.
 - Kibana — visualises and searches logs via dashboards.
 - Used for centralized log management and analysis.
-

Q: What is Datadog?

- Datadog is a cloud-based monitoring and analytics platform.
 - Provides metrics, logs, traces, and APM (Application Performance Monitoring) in one place.
 - Agent installed on hosts sends data to Datadog cloud.
 - Advantage over ELK + Prometheus: all-in-one SaaS solution, less operational overhead.
-

Q: What metrics do you monitor? What alerts have you configured?

- Infrastructure: CPU usage, memory usage, disk I/O, network traffic.
 - Kubernetes: Pod restart count, OOMKilled events, pending pods, node capacity.
 - Application: request rate, error rate (5xx), response latency (p95/p99).
 - Common alerts: CPU > 85% for 5 mins, Pod CrashLoopBackOff, disk > 80%, high error rate.
-

11. Security & DevSecOps

Q: How do you secure AWS accounts?

- Enable MFA on root and all IAM users.
 - Use IAM Roles instead of long-term access keys where possible.
 - Apply the principle of least privilege — only grant permissions needed.
 - Enable CloudTrail for audit logging of all API calls.
 - Enable GuardDuty for threat detection.
 - Enable AWS Config to detect configuration drift.
 - Use SCPs (Service Control Policies) in AWS Organizations to restrict actions.
-

Q: How do you secure Kubernetes clusters?

- Use RBAC to restrict who can access Kubernetes APIs.
 - Use Network Policies to control Pod-to-Pod communication.
 - Scan container images with Trivy before deploying.
 - Do not run containers as root — use non-root users in Dockerfiles.
 - Use Kubernetes Secrets (or Vault) for sensitive data — never hardcode in manifests.
 - Enable audit logging on the Kubernetes API server.
 - Use Pod Security Standards (restricted/baseline profiles).
-

Q: What is DevSecOps?

- DevSecOps = DevOps + Security. Security is integrated throughout the development lifecycle, not bolted on at the end.
 - Shift-Left Security — catch security issues early in development.
 - Tools: SonarQube (SAST), Trivy (container scan), OWASP ZAP (DAST), AWS Security Hub.
 - Goal: security is everyone's responsibility, not just the security team.
-

12. Bash & Python Scripting

Q: Why use Bash scripting in DevOps?

- Bash is the native scripting language of Linux — ideal for automating shell commands.
 - Used for: server setup scripts, log rotation, backup automation, CI/CD pipeline steps.
 - Runs on any Linux system without installing extra dependencies.
-

Q: Explain loops and functions in Bash.

- for loop: `for i in 1 2 3; do echo $i; done`
 - while loop: `while [ condition ]; do ... done`
 - Function: `my_func() { echo 'Hello'; }` — call with: `my_func`
 - Check exit code: `if [ $? -eq 0 ]; then echo 'success'; fi`
-

Q: Why use Python in DevOps? When to choose Python over Bash?

- Python is better for complex logic: parsing JSON/YAML, calling APIs, data processing.
 - Libraries: boto3 (AWS SDK), requests (HTTP), paramiko (SSH), subprocess (shell commands).
 - Choose Bash for simple shell command automation. Choose Python for anything involving APIs, loops with data, or error handling.
 - boto3 example: `ec2 = boto3.client('ec2'); instances = ec2.describe_instances()`
-

13. Troubleshooting Scenarios

Q: Scenario: Application deployed successfully but is inaccessible.

- Check if Pods are Running: `kubectl get pods`
 - Check Service exists and has the right selector: `kubectl describe svc`
 - Check Ingress rules: `kubectl describe ingress`
 - Check DNS — is the domain resolving to the correct IP?
 - Check Security Groups / NACLs — is the port open?
 - Test from inside the cluster: `kubectl exec -it -- curl http://svc-name`
-

Q: Scenario: CI/CD Pipeline suddenly fails.

- Check the pipeline logs — identify which stage/step failed.
 - Common causes: test failure, Docker build error, missing secret/credential, registry push failure.
 - Check if the runner/agent is healthy.
 - Check if environment variables or secrets changed.
 - Verify the base image is accessible.
-

Q: Scenario: EKS application becomes slow.

- Check Pod metrics: `kubectl top pods` — is CPU or memory maxed out?
 - Check HPA: `kubectl get hpa` — is it scaling?
 - Check for throttled CPU (requests too low): describe pod and look at resource limits.
 - Check application logs for slow queries or errors.
 - Check Node resource usage: `kubectl top nodes`
 - Check ALB / Ingress access logs for high response times.
-

Q: Scenario: Production deployment fails. How do you rollback?

- `kubectl rollout undo deployment/myapp` — rolls back to the previous ReplicaSet.
 - Confirm rollback: `kubectl rollout status deployment/myapp`
 - If using Helm: `helm rollback`
 - Alert the team, check logs for root cause, fix the issue before re-deploying.
 - Post-incident: write a blameless post-mortem.
-

Q: Scenario: Terraform apply fails. How do you debug?

- Read the error message carefully — Terraform usually tells you exactly what failed.
 - Common causes: missing IAM permissions, resource limit exceeded, dependency error, state lock.
 - Unlock state if stuck: `terraform force-unlock`
 - Run `terraform plan` first to validate before applying.
 - Use `terraform state list` to inspect current state.
 - Use `TF_LOG=DEBUG terraform apply` for verbose output.
-

14. System Design

Q: Design a CI/CD pipeline for Kubernetes.

- Developer pushes code to GitHub feature branch.
 - Pull Request triggers GitHub Actions workflow.
 - Steps: Checkout → SonarQube scan → Unit Tests → Docker Build → Trivy scan → Push to ECR.
 - On merge to main: Deploy to staging EKS via Helm (kubectl apply or helm upgrade).
 - Manual approval gate → Deploy to production EKS.
 - Monitoring alerts verify deployment health.
-

Q: Design a highly available AWS application.

- Multi-AZ VPC with public and private subnets across at least 2 AZs.
 - Application Load Balancer (public subnet) distributing traffic to Auto Scaling Group (private subnet).
 - RDS with Multi-AZ for database high availability.
 - S3 for static assets, CloudFront as CDN.
 - Route53 with health checks for DNS failover.
 - CloudWatch alarms + SNS for alerting.
-

Q: Design a monitoring stack.

- Metrics: Prometheus (scrape) + Grafana (dashboards) + Alertmanager (alerts to Slack/PagerDuty).
 - Logs: Fluent Bit (collect) → Elasticsearch → Kibana (search & visualise).
 - Traces: AWS X-Ray or Jaeger for distributed tracing.
 - Uptime: Blackbox Exporter or Datadog Synthetics for endpoint monitoring.
 - Dashboards: Node, Pod, Application, and Business metrics.
-

Q: Design a multi-environment Terraform setup.

- Directory structure: environments/dev/, environments/staging/, environments/prod/.
 - Each environment has its own tfvars file for different sizes/configurations.
 - Shared modules in modules/ directory (vpc, eks, rds, etc.).
 - Each environment has its own remote state in S3 with a separate key (e.g., dev/terraform.tfstate).
 - Use Terraform workspaces or separate directories — separate directories preferred for isolation.
-

15. Behavioral Questions

Q: Tell me about yourself.

- Structure: Present → Past → Future.
 - Present: I am a DevOps Engineer intern with hands-on experience in AWS, Kubernetes, Docker, Terraform, and CI/CD pipelines.
 - Past: During my internship, I worked on [your project] — I built CI/CD pipelines using Jenkins/GitHub Actions, provisioned infrastructure with Terraform, and deployed applications on EKS.
 - Future: I am looking for a full-time role where I can continue growing in cloud and DevOps engineering.
-

Q: Describe a challenging technical issue you solved.

- Use the STAR method: Situation → Task → Action → Result.
 - Situation: Describe the context (production issue, failed deployment, performance problem).
 - Task: What was your responsibility.
 - Action: What steps did you take to debug and resolve.
 - Result: Quantify the impact (reduced downtime by X%, pipeline runtime cut by Y%).
-

Q: Tell me about a time you automated a manual process.

- Example: Previously, deployment was manual — SSH into server, git pull, restart service.
 - I automated it by creating a Jenkins pipeline that builds the Docker image, pushes to ECR, and deploys via kubectl.
 - Result: Deployment time reduced from 30 minutes to 5 minutes, zero manual errors.
-

Q: How do you learn new technologies quickly?

- I start with the official documentation to understand core concepts.
 - I set up a hands-on lab environment (AWS free tier, Minikube, or Terraform Cloud) to practice.
 - I follow structured paths on platforms like A Cloud Guru, Udemy, or Kubernetes documentation.
 - I apply the knowledge to a real project or side project immediately.
-

Q: Where do you see yourself in 5 years?

- I want to grow into a Senior DevOps/Cloud Engineer or SRE role.
 - I aim to gain deeper expertise in cloud architecture, platform engineering, and reliability engineering.
 - I would like to lead DevOps initiatives, mentor junior engineers, and contribute to defining engineering standards.
-

16. Quick Reference: Tools & Commands

Tool / Command	Purpose	Example
kubectl get pods	List Pods	kubectl get pods -n default
kubectl describe	Detailed info + events	kubectl describe pod mypod
kubectl logs	View Pod logs	kubectl logs mypod --previous
kubectl rollout undo	Rollback deployment	kubectl rollout undo deploy/app
kubectl top	Resource usage	kubectl top pods
docker logs	Container logs	docker logs <container-id>
docker exec	Shell into container	docker exec -it <id> /bin/sh
terraform plan	Preview changes	terraform plan -out=plan.tfplan
terraform apply	Apply changes	terraform apply plan.tfplan
terraform state list	List managed resources	terraform state list
df -h	Disk usage	df -h /var
top / htop	CPU & memory usage	top -u www-data
netstat / ss	Network connections	ss -tulnp
journalctl	System logs	journalctl -u nginx --since today